

# Processamento paralelo de lista encadeada com `task`

Eugênio Vitor Lopes dos Santos

DCA3703 – Programação Paralela, Universidade Federal do Rio Grande do Norte

## 1 Enunciado

Implementar em C um programa que cria uma lista encadeada com nós contendo nomes de arquivos fictícios. Dentro de uma região paralela, o programa deve percorrer a lista e criar uma tarefa com `#pragma omp task` para processar cada nó. Cada tarefa imprime o nome do arquivo e o identificador da thread que a executou (`omp_get_thread_num`). Após executar o programa, deve-se refletir: todos os nós foram processados? Algum foi processado mais de uma vez ou ignorado? O comportamento muda entre execuções? Como garantir que cada nó seja processado uma única vez e por apenas uma tarefa?

## 2 Fundamentação teórica

### 2.1 Modelo de tarefas do OpenMP

O OpenMP oferece o modelo de tarefas (`task`) para expressar paralelismo em estruturas de dados irregulares, como listas encadeadas, árvores e grafos. Diferentemente de `parallel for`, que exige que o número de iterações seja conhecido antes da execução, as tarefas podem ser criadas dinamicamente conforme a estrutura é percorrida.

Uma diretiva `#pragma omp task` delimita um bloco de código que pode ser executado por qualquer thread do time. A thread que criou a tarefa continua o fluxo normal; a tarefa entra em uma fila e qualquer thread pode executá-la. Se a thread criadora não tiver trabalho imediato e houver tarefas pendentes, ela pode executá-las. Caso contrário, threads ociosas podem “roubar” tarefas da fila de outras, mecanismo conhecido como *work-stealing* (roubo de trabalho).

### 2.2 As diretivas `single` e `task`

- `#pragma omp single`: delimita um bloco que deve ser executado por exatamente uma thread do time. As demais threads saltam o bloco e prosseguem. É o mecanismo correto para que apenas uma thread percorra a lista e crie as tarefas.
- `#pragma omp task`: cria uma tarefa com o bloco de código seguinte. A tarefa pode ser executada imediatamente ou postergada, dependendo do escalonador.
- `#pragma omp taskwait`: barreira que aguarda a conclusão de todas as tarefas criadas dentro do escopo atual. É necessária quando o código após ela depende do resultado das tarefas.

### 2.3 Escopo de variáveis em tarefas

Uma variável referenciada dentro de uma `task` segue as mesmas regras de escopo da região `parallel` que a contém, a menos que uma cláusula de escopo seja explicitada:

- `shared(var)`: todas as tarefas compartilham o mesmo endereço de memória. Se a variável muda após a criação da tarefa, a tarefa vê o novo valor, o que, no caso de um ponteiro de travessia, faz todas as tarefas enxergarem o mesmo nó (ou NULL).
- `firstprivate(var)`: cada tarefa recebe uma cópia privada inicializada com o valor da variável no momento da criação. É a cláusula adequada para capturar o ponteiro atual em cada iteração de um laço.
- `private(var)`: cria uma cópia privada sem inicialização (valor indefinido). Causa comportamento imprevisível.

### 3 Implementação

Os arquivos-fonte `e0.c` a `e5.c` apresentam o modelo de tarefas de forma incremental. As versões com erro `e1` e `e2` têm o mesmo problema; mantemos `e2` por incluir o contador que quantifica as repetições. As funções auxiliares (`criar_no`, `inserir_no_fim`, `liberar_lista`) e a estrutura `No` são idênticas em todas as versões; por isso, os trechos a seguir mostram apenas a `main()` de cada versão, onde as diretivas OpenMP e as cláusulas de escopo variam. O código completo de cada arquivo encontra-se no Apêndice A.

Compilamos os códigos-fonte com GCC 13.3, padrão C11, avisos ativados (`-Wall`) e suporte a OpenMP (`-fopenmp`).

```

1 gcc -std=c11 -Wall -fopenmp e0.c -o e0
2 gcc -std=c11 -Wall -fopenmp e2.c -o e2
3 gcc -std=c11 -Wall -fopenmp e3.c -o e3
4 gcc -std=c11 -Wall -fopenmp e4.c -o e4
5 gcc -std=c11 -Wall -fopenmp e5.c -o e5
6
7 OMP_NUM_THREADS=4 ./e2
8 OMP_NUM_THREADS=4 ./e5

```

**Listing 1:** Compilação e execução dos programas

#### 3.1 Versão base sequencial (e0.c)

Constrói a lista encadeada com 5 nomes de arquivo e a percorre sequencialmente, sem qualquer diretiva OpenMP. Serve como referência para o comportamento esperado.

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4
5 typedef struct no {
6     char nome_arquivo[128];
7     struct no *next;
8 } No;
9
10 No *criar_no(const char *nome_arquivo) {

```

```

11     No *novo_no = malloc(sizeof(No));
12     if (!novo_no) {
13         perror("malloc");
14         exit(EXIT_FAILURE);
15     }
16     strncpy(novo_no->nome_arquivo, nome_arquivo, sizeof(novo_no->
17         nome_arquivo) - 1);
18     novo_no->nome_arquivo[sizeof(novo_no->nome_arquivo) - 1] = '\0';
19     novo_no->next = NULL;
20     return novo_no;
21 }
22
23 void inserir_no_fim(No **cabeca, const char *nome_arquivo) {
24     No *novo_no = criar_no(nome_arquivo);
25     if (*cabeca == NULL) {
26         *cabeca = novo_no;
27         return;
28     }
29     No *atual = *cabeca;
30     while (atual->next != NULL) {
31         atual = atual->next;
32     }
33     atual->next = novo_no;
34 }
35
36 void liberar_lista(No *cabeca) {
37     No *atual = cabeca;
38     while (atual != NULL) {
39         No *temp = atual;
40         atual = atual->next;
41         free(temp);
42     }
43 }
44
45 int main() {
46     No *cabeca = NULL;
47
48     inserir_no_fim(&cabeca, "documento1.txt");
49     inserir_no_fim(&cabeca, "imagem2.png");
50     inserir_no_fim(&cabeca, "dados3.csv");
51     inserir_no_fim(&cabeca, "relatorio4.pdf");
52     inserir_no_fim(&cabeca, "backup5.zip");
53
54     No *atual = cabeca;
55     while (atual != NULL) {
56         printf("Arquivo: %s\n", atual->nome_arquivo);
57         atual = atual->next;
58     }

```

```

59     liberar_lista(cabeca);
60     return 0;
61 }

```

**Listing 2:** e0.c, versão sequencial base

### 3.2 Erro 1: ausência de `single` (e2.c)

O laço de travessia fica diretamente dentro de `#pragma omp parallel`. Todas as  $T$  threads executam o laço completo e cada uma cria  $N$  tarefas, totalizando  $N \times T$  tarefas e  $N \times T$  processamentos do mesmo nó. O contador processados quantifica o excesso.

```

1  int main() {
2      No *cabeca = NULL;
3      /* cria lista com 5 n00f3s */
4
5      int nos_lista = 5;
6      int processados = 0;
7
8      #pragma omp parallel
9      {
10         No *atual = cabeca;
11         while (atual != NULL) {
12             #pragma omp task
13             {
14                 printf("  [Thread %d] %s\n",
15                     omp_get_thread_num(), atual->nome_arquivo);
16                 processados++;
17             }
18             atual = atual->next;
19         }
20     }
21     #pragma omp taskwait
22 }
23
24 printf("Nos na lista (N): %d\n"
25        "Processamentos totais (Y): %d\n"
26        "Relacao: Y = N x T = %d x %d = %d (esperado)\n",
27        nos_lista, processados,
28        nos_lista, (processados / nos_lista), processados);
29
30 liberar_lista(cabeca);
31 return 0;
32 }

```

**Listing 3:** main() de e2.c, parallel sem single

### 3.3 Erro 2: `single` sem `firstprivate` (e3.c)

Há `#pragma omp single` para que apenas uma thread crie as tarefas. A cláusula `shared(atual)` faz todas as tarefas compartilharem o mesmo ponteiro. Como o laço avança o ponteiro imediatamente após criar cada tarefa, as tarefas executadas posteriormente leem um valor já modificado (outro nó ou NULL).

```
1 int main() {
2     No *cabeca = NULL;
3     inserir_no_fim(&cabeca, "doc1.txt");
4
5     int nos_lista = 1;
6     int processados = 0;
7
8     #pragma omp parallel
9     {
10         #pragma omp single
11         {
12             No *atual = cabeca;
13             while (atual != NULL) {
14                 #pragma omp task shared(atual)
15                 {
16                     printf(" [Thread %d] atual aponta para: %s\n",
17                           omp_get_thread_num(),
18                           atual ? atual->nome_arquivo : "(NULL)");
19                     processados++;
20                 }
21                 atual = atual->next;
22             }
23         }
24     }
25
26     printf("Nos na lista: %d | Processamentos contados: %d\n",
27           nos_lista, processados);
28
29     liberar_lista(cabeca);
30     return 0;
31 }
```

**Listing 4:** `main()` de e3.c, `single` + `shared`

### 3.4 Erro 3: `firstprivate` sem `taskwait` (e4.c)

As tarefas recebem `firstprivate(atual)` e cada uma captura o endereço correto do nó — o nome impresso é sempre correto. No entanto, sem `taskwait`, não há garantia formal de que as tarefas estejam concluídas antes que código subsequente dependa de seus resultados. A barreira implícita no fim da região `parallel` compensa neste caso específico, mas não é a forma portátil de sincronizar.

```
1 int main() {
```

```

2      No *cabeca = NULL;
3      inserir_no_fim(&cabeca, "doc1.txt");
4
5      int nos_lista = 1;
6      int processados = 0;
7
8      #pragma omp parallel
9      {
10         #pragma omp single
11         {
12             No *atual = cabeca;
13             while (atual != NULL) {
14                 #pragma omp task firstprivate(atual)
15                 {
16                     printf(" [Thread %d] Arquivo: %s\n",
17                            omp_get_thread_num(), atual->nome_arquivo);
18                     processados++;
19                 }
20                 atual = atual->next;
21             }
22         }
23     }
24
25     printf("Nos na lista: %d | Processamentos contados: %d\n",
26           nos_lista, processados);
27
28     liberar_lista(cabeca);
29     return 0;
30 }

```

**Listing 5:** main() de e4.c, firstprivate sem taskwait

### 3.5 Versão correta (e5.c)

A solução combina três elementos:

1. single: apenas uma thread percorre a lista e cria as tarefas;
2. firstprivate(atual): cada tarefa captura o endereço do nó no momento da criação;
3. taskwait: sincroniza o término das tarefas antes de prosseguir.

```

1 int main() {
2     No *cabeca = NULL;
3     inserir_no_fim(&cabeca, "doc1.txt");
4
5     int nos_lista = 1;
6     int processados = 0;
7
8     #pragma omp parallel

```

```

9      {
10         #pragma omp single
11         {
12             No *atual = cabeca;
13             while (atual != NULL) {
14                 #pragma omp task firstprivate(atual)
15                 {
16                     printf("  [Thread %d/%d] Arquivo: %s\n",
17                           omp_get_thread_num(),
18                           omp_get_num_threads(),
19                           atual->nome_arquivo);
20                     processados++;
21                 }
22                 atual = atual->next;
23             }
24         }
25     }
26
27     printf("Nos na lista: %d | Processamentos: %d (devem ser iguais!)\n",
28           nos_lista, processados);
29
30     liberar_lista(cabeca);
31     return 0;
32 }

```

**Listing 6:** main() de e5.c, versão correta

## 4 Resultados e discussão

A Tabela 1 resume o comportamento de cada versão com 4 threads.

**Table 1:** Comparação do processamento das versões com 4 threads

| Arquivo | Nós process.        | Nome correto?   | Problema                               |
|---------|---------------------|-----------------|----------------------------------------|
| e0.c    | 5 (sequencial)      | Sim             | Sem paralelismo                        |
| e2.c    | 20 ( $N \times T$ ) | Sim             | Contador quantifica o excesso          |
| e3.c    | 5                   | Sim (por acaso) | shared(current): ponteiro inválido     |
| e4.c    | 5                   | Sim             | Sem taskwait: sem sincronização formal |
| e5.c    | 5                   | Sim             | Correto: single+firstprivate+taskwait  |

### 4.1 Ausência de single (e2.c)

Com 4 threads e 5 nós, e2.c confirma  $Y = 20$ :

```
1 $ OMP_NUM_THREADS=4 ./e2
2 [Thread 0] doc1.txt
3 [Thread 0] img2.png
4 [Thread 0] dat3.csv
5 [Thread 0] rep4.pdf
6 [Thread 0] bkp5.zip
7 [Thread 0] doc1.txt
8 [Thread 0] img2.png
9 [Thread 0] dat3.csv
10 [Thread 3] bkp5.zip
11 [Thread 3] doc1.txt
12 [Thread 3] img2.png
13 [Thread 3] dat3.csv
14 [Thread 3] rep4.pdf
15 [Thread 3] bkp5.zip
16 [Thread 0] rep4.pdf
17 [Thread 3] doc1.txt
18 [Thread 0] img2.png
19 [Thread 3] dat3.csv
20 [Thread 1] rep4.pdf
21 [Thread 1] bkp5.zip
22 Nos na lista (N): 5
23 Processamentos totais (Y): 20
24 Relacao: Y = N x T = 5 x 4 = 20 (esperado)
```

**Listing 7:** Saída de e2.c com 4 threads confirmando  $N \times T$

Cada thread executou o laço independentemente e gerou 5 tarefas. O programa gastou  $T$  vezes mais trabalho que o necessário. Se as tarefas tivessem efeitos colaterais (escrita em arquivo, atualização de banco de dados), cada nó seria alterado  $T$  vezes.



## 4.2 Compartilhamento do ponteiro (e3.c)

Mesmo com `single`, se a cláusula de escopo é `shared(current)`, o ponteiro avança enquanto as tarefas ainda não executaram. O comportamento é não-determinístico: o contador acusa o número correto de nós, mas algumas tarefas podem imprimir NULL ou nomes trocados, dependendo do instante em que o escalonador as ativa.

Sem `firstprivate`, a correção do resultado depende de uma condição de corrida favorável. Não há garantia alguma. Comparando `e3.c` (`shared`) com `e4.c` (`firstprivate`), vê-se que a cláusula de escopo é decisiva.

## 4.3 Ausência de `taskwait` (e4.c)

Com `firstprivate`, `e4.c` imprime os 5 nomes corretos — cada tarefa captura o valor do ponteiro atual no instante da criação, então o laço `atual = atual->next` não afeta tarefas já criadas. No entanto, sem `taskwait`, não há sincronização formal entre a criação das tarefas e código subsequente que dependa delas. Neste caso, a barreira implícita no fim da região `parallel` compensa, mas o uso de `taskwait` é a forma correta e portátil de garantir a conclusão.

## 4.4 Versão correta

Com `single`, `firstprivate(atual)` e `taskwait` (código análogo ao de `e4.c` acrescentando a barreira), cada nó é processado exatamente uma vez, como mostra a saída de `e5.c`:

```
1 $ OMP_NUM_THREADS=4 ./e5
2 [Thread 1/4] Arquivo: doc1.txt
3 [Thread 1/4] Arquivo: bkp5.zip
4 [Thread 2/4] Arquivo: img2.png
5 [Thread 3/4] Arquivo: dat3.csv
6 [Thread 0/4] Arquivo: rep4.pdf
7 Nos na lista: 5 | Processamentos: 5 (devem ser iguais!)
```

**Listing 8:** Execução da versão correta com 4 threads

O programa imprimiu 5 linhas (uma por nó). Quatro threads diferentes executaram as tarefas. A ordem varia entre execuções, mas a contagem permanece 5.

## 5 Conclusão

O paralelismo de listas encadeadas com OpenMP exige três elementos:

1. `single`: restringe a criação de tarefas a uma única thread, evitando a multiplicação de trabalho;
2. `firstprivate`: captura o valor do ponteiro de travessia no momento da criação, impedindo que tarefas compartilhem um endereço que avança;
3. `taskwait`: sincroniza o término das tarefas antes de prosseguir.

A ausência de qualquer um desses elementos resulta em um dos três erros: (i) cada nó é processado  $T$  vezes (sem `single`), (ii) tarefas leem ponteiros inválidos ou trocados (sem `firstprivate`) ou (iii) não há garantia de conclusão antes de código subsequente (sem `taskwait`). O comportamento varia entre execuções porque o escalonamento de tarefas é não-determinístico: a thread que executa cada tarefa, a ordem de execução e o instante de ativação dependem do estado do runtime.

## A Código-fonte completo

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4
5 typedef struct no {
6     char nome_arquivo[128];
7     struct no *next;
8 } No;
9
10 No *criar_no(const char *nome_arquivo) {
11     No *novo_no = malloc(sizeof(No));
12     if (!novo_no) {
13         perror("malloc");
14         exit(EXIT_FAILURE);
15     }
16     strncpy(novo_no->nome_arquivo, nome_arquivo, sizeof(novo_no->
17         nome_arquivo) - 1);
18     novo_no->nome_arquivo[sizeof(novo_no->nome_arquivo) - 1] = '\0';
19     novo_no->next = NULL;
20     return novo_no;
21 }
22
23 void inserir_no_fim(No **cabeca, const char *nome_arquivo) {
24     No *novo_no = criar_no(nome_arquivo);
25     if (*cabeca == NULL) {
26         *cabeca = novo_no;
27         return;
28     }
29     No *atual = *cabeca;
30     while (atual->next != NULL) {
31         atual = atual->next;
32     }
33     atual->next = novo_no;
34 }
35
36 void liberar_lista(No *cabeca) {
37     No *atual = cabeca;
38     while (atual != NULL) {
39         No *temp = atual;
40         atual = atual->next;
41         free(temp);
42     }
43 }
44
45 int main() {
46     No *cabeca = NULL;
```

```

47     inserir_no_fim(&cabeca, "documento1.txt");
48     inserir_no_fim(&cabeca, "imagem2.png");
49     inserir_no_fim(&cabeca, "dados3.csv");
50     inserir_no_fim(&cabeca, "relatorio4.pdf");
51     inserir_no_fim(&cabeca, "backup5.zip");
52
53     No *atual = cabeca;
54     while (atual != NULL) {
55         printf("Arquivo: %s\n", atual->nome_arquivo);
56         atual = atual->next;
57     }
58
59     liberar_lista(cabeca);
60     return 0;
61 }

```

**Listing 9:** e0.c, versão sequencial base

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4  #include <omp.h>
5
6  typedef struct no {
7      char nome_arquivo[128];
8      struct no *next;
9  } No;
10
11 No *criar_no(const char *nome_arquivo) {
12     No *no = malloc(sizeof(No));
13     strncpy(no->nome_arquivo, nome_arquivo, sizeof(no->nome_arquivo) - 1);
14     no->nome_arquivo[sizeof(no->nome_arquivo) - 1] = '\0';
15     no->next = NULL;
16     return no;
17 }
18
19 void inserir_no_fim(No **cabeca, const char *nome_arquivo) {
20     No *novo_no = criar_no(nome_arquivo);
21     if (*cabeca == NULL) {
22         *cabeca = novo_no;
23         return;
24     }
25     No *atual = *cabeca;
26     while (atual->next != NULL) {
27         atual = atual->next;
28     }
29     atual->next = novo_no;
30 }
31
32 void liberar_lista(No *cabeca) {

```

```

33     while (cabeca) {
34         No *temp = cabeca;
35         cabeca = cabeca->next;
36         free(temp);
37     }
38 }
39
40 int main() {
41     No *cabeca = NULL;
42     inserir_no_fim(&cabeca, "doc1.txt");
43     inserir_no_fim(&cabeca, "img2.png");
44     inserir_no_fim(&cabeca, "dat3.csv");
45     inserir_no_fim(&cabeca, "rep4.pdf");
46     inserir_no_fim(&cabeca, "bkp5.zip");
47
48     int nos_lista = 5;
49     int processados = 0;
50
51     #pragma omp parallel
52     {
53         No *atual = cabeca;
54         while (atual != NULL) {
55             #pragma omp task
56             {
57                 printf("  [Thread %d] %s\n",
58                     omp_get_thread_num(), atual->nome_arquivo);
59
60                 processados++;
61             }
62             atual = atual->next;
63         }
64     }
65
66     printf("Nos na lista (N): %d\nProcessamentos totais (Y): %d\nRelacao:
67         Y = N x T = %d x %d = %d (esperado)\n",
68         nos_lista, processados, nos_lista, (processados / nos_lista),
69         processados);
70
71     liberar_lista(cabeca);
72     return 0;
73 }

```

**Listing 10:** e2.c, parallel sem single com contador

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #include <omp.h>
5
6

```

```

7 typedef struct no {
8     char nome_arquivo[128];
9     struct no *next;
10 } No;
11
12 No *criar_no(const char *nome_arquivo) {
13     No *no = malloc(sizeof(No));
14     strncpy(no->nome_arquivo, nome_arquivo, sizeof(no->nome_arquivo) - 1);
15     no->nome_arquivo[sizeof(no->nome_arquivo) - 1] = '\0';
16     no->next = NULL;
17     return no;
18 }
19
20 void inserir_no_fim(No **cabeca, const char *nome_arquivo) {
21     No *novo_no = criar_no(nome_arquivo);
22     if (*cabeca == NULL) {
23         *cabeca = novo_no;
24         return;
25     }
26     No *atual = *cabeca;
27     while (atual->next != NULL) {
28         atual = atual->next;
29     }
30     atual->next = novo_no;
31 }
32
33 void liberar_lista(No *cabeca) {
34     while (cabeca) {
35         No *temp = cabeca;
36         cabeca = cabeca->next;
37         free(temp);
38     }
39 }
40
41 int main() {
42     No *cabeca = NULL;
43     inserir_no_fim(&cabeca, "doc1.txt");
44     inserir_no_fim(&cabeca, "img2.png");
45     inserir_no_fim(&cabeca, "dat3.csv");
46     inserir_no_fim(&cabeca, "rep4.pdf");
47     inserir_no_fim(&cabeca, "bkp5.zip");
48
49     int nos_lista = 15;
50     int processados = 0;
51
52     #pragma omp parallel
53     {
54         #pragma omp single
55         {

```

```

56         No *atual = cabeca;
57         while (atual != NULL) {
58             #pragma omp task shared(atual)
59             {
60                 printf(" [Thread %d] atual aponta para: %s\n",
61                     omp_get_thread_num(),
62                     atual ? atual->nome_arquivo : "(NULL)");
63                 processados++;
64             }
65             atual = atual->next;
66         }
67     }
68 }
69
70 printf("Nos na lista: %d | Processamentos contados: %d\n",
71     nos_lista, processados);
72
73 liberar_lista(cabeca);
74 return 0;
75 }

```

**Listing 11:** e3.c, single sem firstprivate

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4  #include <omp.h>
5
6
7  typedef struct no {
8      char nome_arquivo[128];
9      struct no *next;
10 } No;
11
12 No *criar_no(const char *nome_arquivo) {
13     No *no = malloc(sizeof(No));
14     strncpy(no->nome_arquivo, nome_arquivo, sizeof(no->nome_arquivo) - 1);
15     no->nome_arquivo[sizeof(no->nome_arquivo) - 1] = '\0';
16     no->next = NULL;
17     return no;
18 }
19
20 void inserir_no_fim(No **cabeca, const char *nome_arquivo) {
21     No *novo_no = criar_no(nome_arquivo);
22     if (*cabeca == NULL) {
23         *cabeca = novo_no;
24         return;
25     }
26     No *atual = *cabeca;
27     while (atual->next != NULL) {

```

```

28     atual = atual->next;
29 }
30 atual->next = novo_no;
31 }
32
33 void liberar_lista(No *cabeca) {
34     while (cabeca) {
35         No *temp = cabeca;
36         cabeca = cabeca->next;
37         free(temp);
38     }
39 }
40
41 int main() {
42     No *cabeca = NULL;
43     inserir_no_fim(&cabeca, "doc1.txt");
44     inserir_no_fim(&cabeca, "img2.png");
45     inserir_no_fim(&cabeca, "dat3.csv");
46     inserir_no_fim(&cabeca, "rep4.pdf");
47     inserir_no_fim(&cabeca, "bkp5.zip");
48
49     int nos_lista = 5;
50     int processados = 0;
51
52     #pragma omp parallel
53     {
54         #pragma omp single
55         {
56             No *atual = cabeca;
57             while (atual != NULL) {
58                 #pragma omp task firstprivate(atual)
59                 {
60                     printf(" [Thread %d] Arquivo: %s\n",
61                            omp_get_thread_num(), atual->nome_arquivo);
62                     processados++;
63                 }
64                 atual = atual->next;
65             }
66         }
67     }
68
69     printf("Nos na lista: %d | Processamentos contados: %d\n",
70           nos_lista, processados);
71
72     liberar_lista(cabeca);
73     return 0;
74 }

```

**Listing 12:** e4.c, firstprivate sem taskwait



```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #include <omp.h>
5
6
7 typedef struct no {
8     char nome_arquivo[128];
9     struct no *next;
10 } No;
11
12 No *criar_no(const char *nome_arquivo) {
13     No *no = malloc(sizeof(No));
14     strncpy(no->nome_arquivo, nome_arquivo, sizeof(no->nome_arquivo) - 1);
15     no->nome_arquivo[sizeof(no->nome_arquivo) - 1] = '\0';
16     no->next = NULL;
17     return no;
18 }
19
20 void inserir_no_fim(No **cabeca, const char *nome_arquivo) {
21     No *novo_no = criar_no(nome_arquivo);
22     if (*cabeca == NULL) {
23         *cabeca = novo_no;
24         return;
25     }
26     No *atual = *cabeca;
27     while (atual->next != NULL) {
28         atual = atual->next;
29     }
30     atual->next = novo_no;
31 }
32
33 void liberar_lista(No *cabeca) {
34     while (cabeca) {
35         No *temp = cabeca;
36         cabeca = cabeca->next;
37         free(temp);
38     }
39 }
40
41 int main() {
42     No *cabeca = NULL;
43     inserir_no_fim(&cabeca, "doc1.txt");
44     inserir_no_fim(&cabeca, "img2.png");
45     inserir_no_fim(&cabeca, "dat3.csv");
46     inserir_no_fim(&cabeca, "rep4.pdf");
47     inserir_no_fim(&cabeca, "bkp5.zip");
48 }

```

```

49  int nos_lista = 5;
50  int processados = 0;
51
52  #pragma omp parallel
53  {
54      #pragma omp single
55      {
56          No *atual = cabeca;
57          while (atual != NULL) {
58              #pragma omp task firstprivate(atual)
59              {
60                  printf("  [Thread %d/%d] Arquivo: %s\n",
61                      omp_get_thread_num(),
62                      omp_get_num_threads(),
63                      atual->nome_arquivo);
64                  processados++;
65              }
66              atual = atual->next;
67          }
68      }
69      #pragma omp taskwait
70  }
71
72  printf("Nos na lista: %d | Processamentos: %d (devem ser iguais!)\n",
73      nos_lista, processados);
74
75  liberar_lista(cabeca);
76  return 0;
77 }

```

**Listing 13:** e5.c, versão correta com single + firstprivate + taskwait